
Topic 4 — Async Research Assistant

name (@ii1ahe)

name (@ii1ahe)

Final Project Defense – September 17, 2026

`github.com/ii1ahe/research-assistant`

The problem in one sentence

What the system does

You ask one research question; the system retrieves evidence from Wikipedia, arXiv and the open web *concurrently*, and returns one answer whose numbered citations point at the sources it actually used.

Inputs

- ▶ a question (non-empty, ≤ 500 chars)
- ▶ sources: wikipedia, arxiv, web
- ▶ a results-per-source limit (1–20, default 3)

Outputs

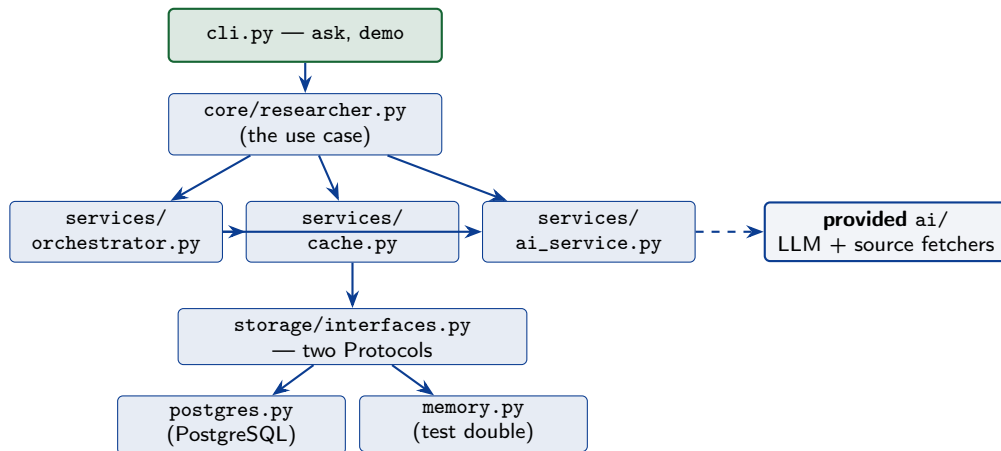
- ▶ a cited answer + a numbered source list
- ▶ a per-source retrieval report
- ▶ a session row, and an exit status

Entry points: `python -m researcher ask` and `demo` (five supplied questions). **No HTTP API** — ADR-006; the container publishes no port.

Exit status is part of the contract: **0** answered (possibly partial), **1** no usable answer, **2** bad input or configuration.

Architecture

Where the provided ai/ ends and our engineering begins.



Layered modular monolith (ADR-001). ai/ is imported, never modified, and reached through one module.

Three design decisions we can defend

1. Protocols over abstract base classes, at both storage seams.

An ABC imposes a *nominal* relationship, so a test double must inherit from the real implementation. A Protocol is structural — which is why `memory.py` is a genuine second implementation with the *same* contract suite run against both.

2. A semaphore, not a bigger gather.

The bound on in-flight sources is configuration (1–16), not the shape of the code. And each task carries its own deadline rather than one wrapping the group — a group deadline lets one slow source destroy two fast results.

3. PostgreSQL for both the cache and the sessions (ADR-002).

One store to operate; ON CONFLICT makes the database enforce the upsert; and PERSIST_SESSIONS=false still gives a database-free run.

Expect to be asked

“Why not threads? Why not SQLite? Why not an HTTP API?” — answers in ADR-005, ADR-002, ADR-006.

Concurrency: the benchmark

Workload: the five supplied sample questions, three sources, three results each, live APIs, fresh cache (`python scripts/bench.py -repeat 1`).

	Sequential (b=1)	Concurrent (b=3)	Speedup
Retrieval (5 questions)	4.83 s	2.89 s	1.67×
Retrieval (per question)	0.97 s	0.58 s	1.67×
End to end (5 questions)	43.4 s	43.3 s	1.00×

Bottleneck named: synthesis is *one sequential model call*, 89 % of wall-clock time. The concurrency design optimised the other 11 %.

What didn't reach $N\times$: the ceiling is not the source count but the ratio of summed to slowest latency — **1.83×** here, so 1.67× is **92 % of the ceiling**.

Robustness: one concrete failure mode

Scenario: mid-verification, Gemini returned 429 RESOURCE_EXHAUSTED (limit: 20, Please retry in 21.9s). Two of five demo questions unanswered.

How we surfaced it: we suspected the new container and *refuted* it — the same question from the host answered in 2.92 s. Then we probed the provider directly to see the raw payload behind our deliberately generic message.

What the system does:

- ▶ **Isolate:** `gather(return_exceptions=True)` + a `SourceOutcome` per source — one question lost, not the run.
- ▶ **Disclose:** the CLI names the failed source; exit 1 separates “some questions failed” from “the program broke”.
- ▶ **Contain:** provider text never reaches a log — it becomes an `UpstreamError` with a safe message.

The gap we are naming, not hiding: we wait a jittered ≤ 0.5 s against a 21.9 s hint, so all three attempts land in one window.

Live demo (or scripted demo)

What we will run

The five supplied sample questions end to end, inside the container, against a real PostgreSQL — showing the answer, its citations, and the per-source report.

Demo command (works inside Docker):

```
$ docker compose up -d postgres
$ docker compose run --rm app
$ docker compose run --rm app python -m researcher ask \
    "What is photosynthesis and what are its main stages?" \
    --sources wikipedia arxiv web
```

- ▶ The per-source report shows attempts=0 on a cache hit, and names the source that failed on a partial run.
- ▶ The session row is written *from inside the container*, 327 tests pass there with **zero skips**.

Backup: the free tier allows 20 syntheses/day/model and a demo costs 5, so a pre-recorded run guards against a spent quota.

Testing

Coverage & shape

- ▶ 327 tests; 96 % coverage over researcher/
- ▶ 5 925 lines of tests vs 4 494 of application code
- ▶ Provided ai/ smoke tests: **16/16 passing**
- ▶ All offline — *enforced*, not intended

Three tests worth calling out

- ▶ Synthesis-deadline regression — *hangs* if the deadline is ever rejoined
- ▶ The offline guard — 11 cases, permits private ranges, refuses public ones
- ▶ The storage contract — one suite, both implementations

Tooling

`pytest + pytest-asyncio (strict) + respx; ruff; mypy -strict` over `researcher/` and `scripts/`. The offline guard exists because five tests once passed while quietly calling the live API — green and wrong is the worst state a test can be in.

Limitations & what we'd do next

Where the system is brittle today

- ▶ **Answer correctness is not verified.** A fluent, correctly-cited, *wrong* answer passes every check we have — we validate structure, never support.
- ▶ **Throttles are honoured, not anticipated.** The provider's Retry-After (measured: 21.9 s) is now parsed at the classification boundary and waited — but the first call still discovers a spent quota by being refused.
- ▶ **Concurrency stops at the question boundary** — the demo's five questions run sequentially, and MAX_PARALLEL_SOURCES bounds sources within a question only.

Given another week we would

1. Add a token bucket per provider, so a quota is anticipated instead of discovered by the first refused call.
2. Stream the synthesis, or overlap it with retrieval — the *only* change that makes the concurrency work visible end to end ($1.67\times \rightarrow$ a real end-to-end win).
3. Move PostgreSQL off trust auth to scram-sha-256, and add a grounding check between each claim and its cited source.

Contributions

Member	Primary ownership	Commits
<i>name</i> (@ii1ahe)	Everything: architecture, services, storage, CLI, tests, container, report	100 %

Solo submission. `git shortlog -sn` on main reports a single author, so the contribution statement is single-member at 100 %.

I can answer for:

Any file in the repo. Pick one — including the four defects that only running the system found.

AI tool use: Claude (Anthropic) for scaffolding, test construction and documentation drafts. Every claim in the report was verified by running the tests, the container and the benchmark; the architectural decisions are mine.

Questions?

name (@ii1ahe)

`github.com/ii1ahe/research-assistant`

"We can walk through any file in the repo."